

PRACA DYPLOMOWA MAGISTERSKA

Analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach

*A performance and efficiency analysis of selected
parallel sorting algorithms on multicore processors*

Krzysztof Pielot

Nr albumu: 133537

Kierunek: Informatyka

Forma studiów: stacjonarne

Poziom studiów: II

Promotor pracy:

dr inż. Bartosz Kowalczyk

Praca przyjęta dnia:

Podpis promotora:

Częstochowa, 2026

Niniejszym chciałbym serdecznie podziękować

...

*za bezcenne wsparcie udzielone mi
w trakcie trwania studiów.*

Spis treści

Wstęp	3
Cel pracy	4
Zakres pracy	4
1 Wprowadzenie teoretyczne	7
1.1 Wprowadzenie do sortowania równoległego	7
1.2 Charakterystyka procesorów wielordzeniowych	7
1.3 Wydajność i efektywność	7
1.4 Skalowalność algorytmów równoległych	7
2 Opis wybranych algorytmów sortowania równoległego	9
2.1 Parallel Quicksort	9
2.2 Parallel Merge Sort	9
2.3 Parallel Bucket Sort	9
2.4 Parallel Samplesort	9
3 Środowisko badawcze i implementacja	11
3.1 Opis środowiska testowego	11
3.2 Wybór języka i bibliotek	12
3.3 Sposób implementacji algorytmów	13
3.4 Charakterystyka danych testowych	13
4 Metodyka badań	15
4.1 Scenariusze testowe	15
4.2 Wariant sekwencyjny jako punkt odniesienia	15
4.3 Parametry pomiarów	15
4.4 Narzędzia do pomiaru czasu, CPU i pamięci	15
4.5 Przebieg eksperymentów	15
5 Analiza wyników badań	17
5.1 Porównanie czasu sortowania	17
5.2 Wpływ liczby rdzeni na wydajność	17
5.3 Analiza zużycia pamięci RAM i obciążenia CPU	17

5.4	Skalowalność algorytmów	17
5.5	Wpływ typu danych	17
5.6	Porównanie efektywności w zależności od charakterystyki danych wej- ściowych	18
	Podsumowanie	19
	Bibliografia	21
	Spis rysunków	23
	Spis tabel	24
	Listing	25
	Streszczenie	27
	Summary	29
	Słowa kluczowe	31

Wstęp

W ostatnim czasie można zaobserwować intensywny rozwój architektury procesorów wielordzeniowych. Producenci sprzętu zamiast dalszego zwiększania częstotliwości taktowania pojedynczego rdzenia decydują się na zwiększanie liczby rdzeni fizycznych oraz logicznych. Taka zmiana podejścia projektowania procesorów stawia przed twórcami oprogramowania nowe wyzwania. Klasyczne algorytmy sekwencyjne nie wykorzystują w pełni dostępnej mocy obliczeniowej współczesnych jednostek centralnych.

Szczególnie ważnym obszarem, w którym równoległość może przynieść wymierne korzyści, jest sortowanie danych. Sortowanie stanowi jeden z fundamentalnych problemów informatyki i jest wykorzystywane w ogromnej liczbie miejsc takich jak bazy danych, systemy plików, aż po przetwarzanie dużych zbiorów danych i uczenie maszynowe. Wraz ze wzrostem objętości przetwarzanych informacji klasyczne algorytmy sortowania sekwencyjnego mogą coraz częściej stać się wąskim gardłem wydajnościowym.

Niestety, implementacja efektywnych algorytmów sortowania równoległego nie jest zadaniem łatwym. Wymaga ona odpowiedniego podziału danych, synchronizacji procesów, zarządzania pamięcią współdzieloną oraz minimalizacji narzutu związanego z tworzeniem i komunikacją procesów. W praktyce wiele istniejących rozwiązań albo pozostaje na poziomie teoretycznym, albo nie uwzględnia realnych ograniczeń sprzętowych i systemowych, takich jak koszt przełączania kontekstu, konflikty w dostępie do pamięci czy nierównomierne obciążenie rdzeni.

W związku z tym, interesującym i aktualnym zagadnieniem wydaje się praktyczne zbadanie wydajności oraz efektywności wybranych algorytmów sortowania równoległego. Niniejsza praca koncentruje się na implementacji i porównaniu czterech popularnych algorytmów: Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort. Algorytmy zostały zrealizowane w języku Python z wykorzystaniem biblioteki `multiprocessing`, a następnie poddane testom pod kątem czasu wykonania, zużycia pamięci RAM, obciążenia procesora, skalowalności oraz wpływu charakterystyki danych wejściowych.

Cel pracy

Głównym celem niniejszej pracy magisterskiej jest przeprowadzenie dogłębnej analizy wydajności i efektywności wybranych algorytmów sortowania równoległego na współczesnych wielordzeniowych procesorach. Praca skupia się na praktycznej implementacji oraz porównaniu czterech algorytmów: Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort.

Kluczowym celem jest zbadanie, w jaki sposób wybrane algorytmy zachowują się w realnych warunkach obliczeniowych, a nie jedynie w warunkach idealnych zakładanych przez modele teoretyczne. W tym celu algorytmy zostały zaimplementowane w języku Python z wykorzystaniem biblioteki `multiprocessing`, która umożliwia tworzenie procesów oraz współdzielenie pamięci. Każdy z algorytmów został uruchomiony zarówno w wariancie sekwencyjnym stanowiącym punkt odniesienia, jak i w wariancie równoległym, przy różnej liczbie wykorzystywanych rdzeni procesora.

Badanie obejmuje kilka kluczowych aspektów:

- ▶ pomiar czasu sortowania w zależności od rozmiaru zbioru danych oraz liczby użytych rdzeni,
- ▶ analizę zużycia pamięci operacyjnej (RAM) oraz obciążenia procesora (CPU) podczas wykonywania algorytmów,
- ▶ ocenę skalowalności, czyli zdolności algorytmów do efektywnego wykorzystania rosnącej liczby rdzeni,
- ▶ porównanie zachowania algorytmów przy różnych charakterystykach danych wejściowych tj. dane losowe, częściowo posortowane oraz z duplikatami.

Dodatkowym celem pracy jest wyznaczenie i analiza klasycznych metryk równoległości, *speedup* (przyspieszenie) oraz *efficiency* (efektywność), a także identyfikacja głównych czynników ograniczających wydajność poszczególnych algorytmów (m.in. narzut związany z tworzeniem procesów, synchronizacją, komunikacją oraz zarządzaniem pamięcią współdzieloną).

W rezultacie pracy oczekuje się uzyskania praktycznych wniosków dotyczących tego, które z badanych algorytmów sortowania równoległego najlepiej sprawdzają się w środowisku wielordzeniowym przy określonych rozmiarach i typach danych, a także wskazania ich mocnych i słabych stron w kontekście rzeczywistego wykorzystania zasobów obliczeniowych.

Zakres pracy

Zakres pracy obejmuje:

- ▶ zebranie wiadomości z zakresu sortowania równoległego, architektury procesorów wielordzeniowych, metryk wydajności i efektywności oraz skalowalności algorytmów równoległych,
- ▶ porównanie wybranych algorytmów sortowania równoległego (Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort) pod kątem wydajności, skalowalności i zużycia zasobów obliczeniowych,
- ▶ opracowanie założeń projektowych dotyczących implementacji algorytmów w środowisku wielordzeniowym z wykorzystaniem biblioteki `multiprocessing` oraz pamięci współdzielonej,
- ▶ implementację wariantów sekwencyjnych i równoległych wybranych algorytmów sortowania w języku Python,
- ▶ implementację generatora danych testowych o różnych charakterystykach (dane losowe, częściowo posortowane, oraz dane z duplikatami),
- ▶ przetestowanie zaimplementowanych algorytmów na wielordzeniowych procesorach, obejmujące pomiar czasu wykonania, zużycia pamięci RAM, obciążenia CPU, wyznaczenie metryk *speedup* i *efficiency* oraz analizę wpływu charakterystyki danych wejściowych.
- ▶ prezentację wyników badań za pomocą tabel oraz wykresów.

W rozdziale 1 przedstawiono podstawy teoretyczne sortowania równoległego, charakterystykę procesorów wielordzeniowych oraz metryki wykorzystywane do oceny wydajności i skalowalności. W rozdziale 2 opisano wybrane algorytmy sortowania równoległego. W rozdziale 3 scharakteryzowano środowisko badawcze, zastosowane technologie oraz sposób implementacji algorytmów. W rozdziale 4 przedstawiono metodykę badań, scenariusze testowe oraz narzędzia pomiarowe. W rozdziale 5 zaprezentowano i przeanalizowano wyniki przeprowadzonych eksperymentów. Pracę zamyka podsumowanie zawierające najważniejsze wnioski oraz możliwe kierunki dalszych badań.

Rozdział 1

Wprowadzenie teoretyczne

- 1.1 Wprowadzenie do sortowania równoległego**
- 1.2 Charakterystyka procesorów wielordzeniowych**
- 1.3 Wydajność i efektywność**
- 1.4 Skalowalność algorytmów równoległych**

Rozdział 2

Opis wybranych algorytmów sortowania równoległego

Dział poświęcony wybranym algorytmom zawierający opis, schemat działania, sposób dzielenia i łączenia, złożoność, potencjalne problemy oraz kiedy algorytm jest efektywny, a kiedy nie

2.1 Parallel Quicksort

2.2 Parallel Merge Sort

2.3 Parallel Bucket Sort

2.4 Parallel Samplesort

Rozdział 3

Środowisko badawcze i implementacja

3.1 Opis środowiska testowego

Wszystkie eksperymenty zostały przeprowadzone na komputerze o następującej konfiguracji:

- ▶ **Procesor:** Intel Core i5-9300H @ 2.40 GHz
- ▶ **Rdzenie fizyczne:** 4
- ▶ **Rdzenie logiczne:** 8
- ▶ **Taktowanie bazowe:** 2401 MHz
- ▶ **Pamięć operacyjna RAM:** 15,85 GB
- ▶ **System operacyjny:** Windows 10
- ▶ **Architektura:** AMD64 (x86-64)
- ▶ **Interpreter Python:** 3.11.7

Środowisko to umożliwia badanie zachowania algorytmów sortowania równoległego przy ograniczonej, ale realnej liczbie rdzeni. Obecność technologii Hyper-Threading (8 procesorów logicznych przy 4 rdzeniach fizycznych) pozwala dodatkowo ocenić wpływ logicznych jednostek wykonawczych na wydajność algorytmów.

Dane dotyczące sprzętu i oprogramowania były automatycznie zebrane i zapisywane w bazie danych SQLite przed przeprowadzaniem eksperymentów.

3.2 Wybór języka i bibliotek

Do implementacji algorytmów sortowania równoległego, genrowania danych, przeprowadzenia eksperymentów oraz wizualizacji wyników wybrano język programowania Python w wersji 3.11.7 [11]. Decyzja ta wynikała z kilku powodów. Python jest jednym z najpopularniejszych języków, charakteryzuje się przejrzystą i elegancką składnią, co ułatwia tworzenie, analizę i utrzymanie kodu. Posiada bogatą bibliotekę standardową oraz rozbudowany zbiór dostępnych bibliotek, pakietów i narzędzi programistycznych rozszerzających możliwości języka, które znacząco przyspieszają rozwój oprogramowania. Dodatkowo język ten jest w pełni przenośny między systemami operacyjnymi tj. Windows, Linux, macOS, co ma znaczenie przy planowaniu ewentualnych dalszych testów na innych platformach.

Niezwyczajnie istotną rolę w projekcie odgrywa biblioteka `multiprocessing` [12], stanowiąca część standardowej biblioteki Pythona. Pakiet ten przy użyciu interfejsu programistycznego podobnego do pythonowego modułu `threading` umożliwia tworzenie procesów oraz realizację obliczeń równoległych z pominięciem ograniczeń wynikających z Global Interpreter Lock (GIL). Dzięki temu możliwe jest efektywne wykorzystanie wielu rdzeni procesora. W ramach tej biblioteki wykorzystano również moduł `multiprocessing.shared_memory` [13], który pozwala na bezpośredni dostęp wielu procesów do wspólnego obszaru pamięci. Mechanizm ten eliminuje konieczność serializacji oraz przesyłania dużych struktur danych między procesami, ograniczając związek z tym narzut, co ma szczególne znaczenie podczas sortowania dużych zbiorów danych.

Biblioteka `NumPy` [14] obsługująca dane numeryczne i szybkie operacje na tablicach, została wykorzystana w optymalizacji algorytmów `Parallel Bucket Sort` oraz `Parallel Samplesort`. Posłużyła w nich do efektywnego rozdzielania elementów do kubeków, co znacznie przyspieszyło etap dystrybucji danych. Ponadto `NumPy` wykorzystano w module analizy złożoności obliczeniowej oraz przy generowaniu wykresów i rankingów.

Wyniki eksperymentów były przechowywane i przetwarzane z użyciem biblioteki `pandas` [15], która oferuje wygodne struktury danych (`DataFrame`) oraz narzędzia do filtrowania, agregacji i analizy tabelarycznej. Do generowania wykresów i wizualizacji wyników wykorzystano bibliotekę `matplotlib` [16].

Do monitorowania zużycia zasobów systemowych wykorzystano bibliotekę `psutil` [17] oraz narzędzia `py-cpuinfo` i `py-spy` [18].

Całość kodu źródłowego została opracowana w środowisku programistycznym `JetBrains PyCharm` [19], które zapewnia wsparcie dla języka Python, debugowanie oraz wygodne zarządzanie projektem.

Wybór powyższego zestawu technologii umożliwia na spójną implementację algorytmów, precyzyjny pomiar ich charakterystyk wydajnościowych oraz wygodną

analizę i prezentację wyników badań.

3.3 Sposób implementacji algorytmów

Opisanie w jaki sposób zostały zaimplementowane algorytmy sortujące. Ogólna architektura, w zależności od algorytmu jak następuje dzielenie, synchronizacja danych, wymiany danych, informacje jak został zaimplementowany waraint sekwencyjny

3.4 Charakterystyka danych testowych

Sposób przechowywania, typy danych(int, float), charakterystyka danych(losowe, częściowo posortowane, z dużą ilością duplikatów), rozmiar testowanych zbiorów danych, jak zostały wygenerowane dane do testów

Rozdział 4

Metodyka badań

4.1 Scenariusze testowe

Opis różnych warunków i przypadków testowych takich jak na jakich rozmiar danych, wariant algorytmu, liczba rdzeni, różne typy zbiorów (losowe itp.)

4.2 Wariant sekwencyjny jako punkt odniesienia

Wyjaśnienie, że dla porównania wyników algorytmów równoległych testowana jest także wersja sekwencyjna każdego algorytmu.

4.3 Parametry pomiarów

Parametry pomiarów co i jak będzie mierzone

4.4 Narzędzia do pomiaru czasu, CPU i pamięci

Opis wykorzystanych narzędzi i bibliotek użytych do pomiarów

4.5 Przebieg eksperymentów

Opis jak zostały przeprowadzone testy. Przygotowanie środowisk testowych. Sposób zapisywania i archiwizacji wyników. Procedura uruchomienia algorytmu itp.

Rozdział 5

Analiza wyników badań

5.1 Porównanie czasu sortowania

Prezentacja wyników pomiarów czasu wykonania dla każdego z algorytmów (dla różnych zbiorów danych, rozmiarów i typów danych). Omówienie i analiza

5.2 Wpływ liczby rdzeni na wydajność

Prezentacja i omówienie wyników pod względem użytych liczby rdzeni

5.3 Analiza zużycia pamięci RAM i obciążenia CPU

Prezentacja i omówienie zużycia pamięci i obciążenia w tym zestawienie średniego i maksymalnego zużycia

5.4 Skalowalność algorytmów

Ocena, jak dobrze algorytmy skalują się wraz ze wzrostem liczby rdzeni i rozmiarem danych

5.5 Wpływ typu danych

Porównanie wyników dla tych samych algorytmów, ale różnych typów danych wejściowych

5.6 Porównanie efektywności w zależności od charakterystyki danych wejściowych

Omówienie wyników dla danych losowych itp. Wskazanie, który algorytm radził sobie lepiej przy danym rodzaju danych

Podsumowanie

Podsumowanie wyników, najważniejsze wnioski, ocena czy cele pracy zostały osiągnięte, ograniczenia przeprowadzonych badań oraz możliwe kierunki dalszego rozwoju i optymalizacji zaimplementowanych algorytmów.

Bibliografia

- [1] Cormen T.H., Leiserson C.E., Rivest R.L., Stein C., *Introduction to Algorithms*, 3rd Edition, The MIT Press, 2009.
- [2] Božidar D., Dobravec T., *Comparison of parallel sorting algorithms*, Technical report, Faculty of Computer and Information Science, University of Ljubljana, November 2015.
- [3] Maus A., *A full parallel Quicksort algorithm for multicore processors*, Department of Informatics, University of Oslo.
- [4] Goodrich M.T., Tamassia R., *Algorithm Design and Applications*, Wiley, 2015.
- [5] Cormen T.H., Leiserson C.E., Rivest R.L., Stein C., *Introduction to Algorithms*, 4th Edition, The MIT Press, 2022.
- [6] Hong H., *Parallel Bucket Sorting Algorithm*, Computer Science Department, San Jose State University.
- [7] Grama A., Gupta A., Karypis G., Kumar V., *Introduction to Parallel Computing*, 2nd Edition, Addison-Wesley, 2003.
- [8] JáJá J., *An Introduction to Parallel Algorithms*, Addison-Wesley, 1992.
- [9] Hennessy J.L., Patterson D.A., *Computer Architecture: A Quantitative Approach*, 4th Edition, Morgan Kaufmann.
- [10] Intel, *Hyper-Threading Technology*, <https://www.intel.com/content/www/us/en/gaming/resothreading.html>, stan na dzień: 08.08.2026.
- [11] Python Software Foundation, *The Python Tutorial*, <https://docs.python.org/3/tutorial/index.html>, stan na dzień: 11.08.2026.
- [12] Python Software Foundation, *multiprocessing — Process-based parallelism*, <https://docs.python.org/3/library/multiprocessing.html>, stan na dzień: 11.08.2026.
- [13]

- [14] NumPy Developers, *NumPy documentation*, <https://numpy.org/doc/stable/>, stan na dzień: 11.08.2026.
- [15] The pandas development team, *pandas documentation*, <https://pandas.pydata.org/docs/>, stan na dzień: 11.08.2026.
- [16] Matplotlib Development Team, *Matplotlib documentation*, <https://matplotlib.org/>, stan na dzień: 11.08.2026.
- [17] Rodola G., *psutil documentation*, <https://psutil.readthedocs.io/en/latest/>, stan na dzień: 11.08.2026.
- [18] Fredrikson B., *py-spy: Sampling profiler for Python programs*, <https://github.com/benfred/py-spy>, stan na dzień: 11.08.2026.
- [19] JetBrains, *PyCharm – The Python IDE*, <https://www.jetbrains.com/pycharm/>, stan na dzień: 11.08.2026.

Spis rysunków

Spis tabel

Spis listingów

Streszczenie

W niniejszej pracy przeprowadzono analizę wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach. W ramach pracy zaimplementowano i porównano algorytmy Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort w wariantach sekwencyjnych i równoległych.

Badania obejmowały pomiary czasu wykonania, zużycia pamięci operacyjnej oraz obciążenia procesora dla różnych rozmiarów danych, liczby rdzeni oraz charakterystyk zbiorów wejściowych. Na podstawie uzyskanych wyników przeanalizowano skalowalność algorytmów oraz wyznaczono wartości przyspieszenia (*speedup*) i efektywności (*efficiency*).

Przeprowadzone eksperymenty pozwoliły na ocenę wpływu równoległości na wydajność poszczególnych algorytmów oraz identyfikację czynników ograniczających ich efektywność, takich jak narzut związany z tworzeniem procesów, synchronizacją, komunikacją i zarządzaniem pamięcią.

Summary

This thesis presents an analysis of the performance and efficiency of selected parallel sorting algorithms on multicore processors. The work implements and compares Parallel Quicksort, Parallel MergeSort, Parallel BucketSort, and Parallel Samplesort in both sequential and parallel variants.

The conducted experiments include measurements of execution time, memory usage, and processor utilization for different data sizes, numbers of processor cores, and characteristics of input datasets. Based on the obtained results, the scalability of the algorithms is analyzed and the speedup and efficiency metrics are determined.

The experiments make it possible to assess the impact of parallelization on the performance of the selected algorithms and to identify factors limiting their efficiency, including the overhead associated with process creation, synchronization, communication, and memory management.

Słowa kluczowe

Algorytmy sortowania, sortowanie równoległe, procesory wielordzeniowe, Quick-sort, Merge Sort, Bucket Sort, Samplesort, programowanie równoległe, wydajność, efektywność, skalowalność, Python, multiprocessing

Imię i nazwisko: Krzysztof Pielot

Nr albumu: 133537

Kierunek: Informatyka

Wydział: Informatyki i Sztucznej Inteligencji

Oświadczenie autora pracy dyplomowej*

Oświadczam pod rygorem odpowiedzialności karnej, że złożona przeze mnie praca dyplomowa pt. „*Analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach*” jest moim samodzielnym opracowaniem i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Jednocześnie oświadczam, że moja praca (w całości ani we fragmentach) nie była wcześniej przedmiotem procedur związanych z uzyskaniem tytułu zawodowego w Politechnice Częstochowskiej.

Wyrażam/~~nie wyrażam~~** zgodę/zgody na nieodpłatne wykorzystanie przez Politechnikę Częstochowską całości lub fragmentów wyżej wymienionej pracy w publikacjach Politechniki Częstochowskiej.

.....
podpis studenta

*W przypadku zbiorowej pracy dyplomowej, dołącza się oświadczenia każdego ze współautorów pracy dyplomowej.

*Niepotrzebne skreślić.